Experiment 17:- Write a Program to Implement all Detecting Outliers Techniques of Machine Learning on any Dataset

Detecting Outliers

If our dataset is small, we can detect the outlier by just looking at the dataset. But what if we have a huge dataset, how do we identify the outliers then? we need to use various visualization and mathematical techniques.

Below are some of the techniques of Detecting outliers

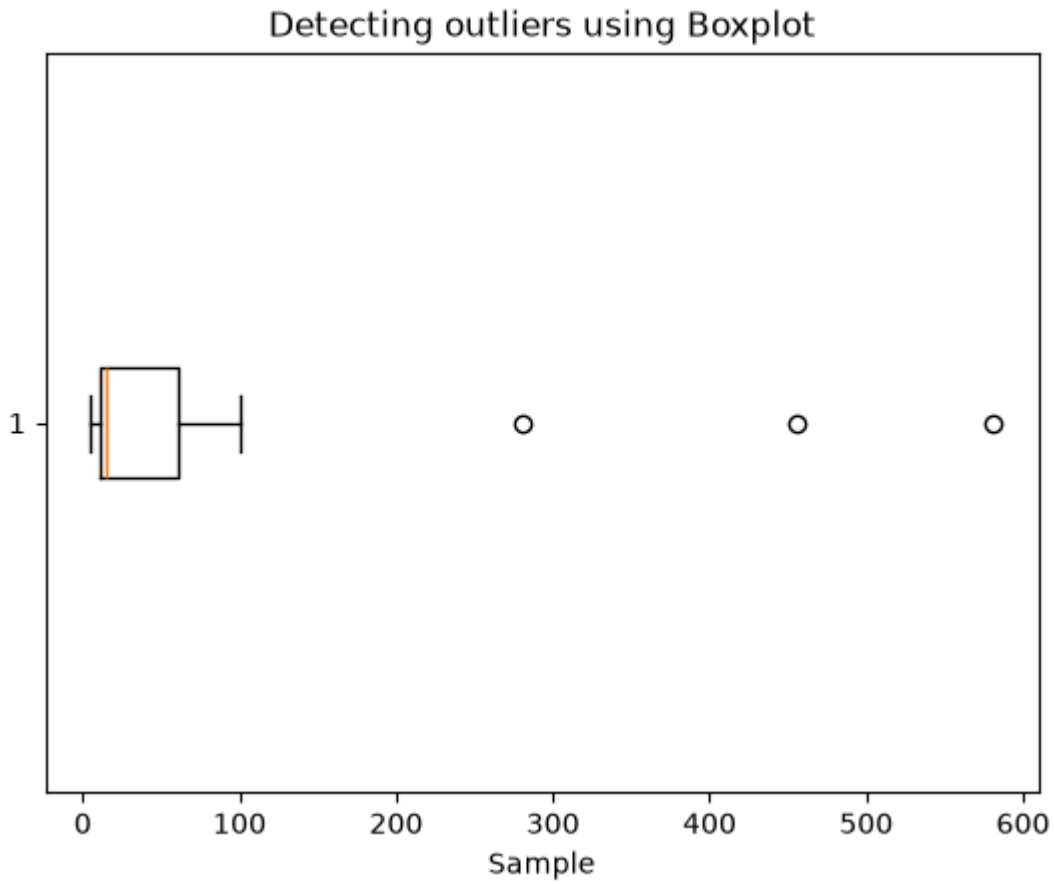
1. Boxplots
2. Z-score
3. Inter Quartile Range(IQR)

1.Detecting outliers using the Boxplots

In [1]:
```
import matplotlib.pyplot as plt

sample = [15, 101 , 18 , 7 , 13 , 16 , 11 , 21 , 5 , 15 , 10 , 9 , 281 , 581 , 45
plt.boxplot(sample , vert= False)
plt.title("Detecting outliers using Boxplot")
plt.xlabel('Sample')
plt.show()
```

```
C:\Users\asus\AppData\Local\Temp\ipykernel_28172\2157929664.py:4: MatplotlibDeprec
ationWarning: vert: bool was deprecated in Matplotlib 3.11 and will be removed in
3.13. Use orientation: {'vertical', 'horizontal'} instead.
  plt.boxplot(sample , vert= False)
```



2.Detecting outliers using the Z-scores

Criteria:

Any data point whose Z-score falls out of 3rd standard deviation is an outlier.

Steps:

1.Loop through all the data points and compute the Z-score using the formula (Xi-mean)/std.

2.Define a threshold value of 3 and mark the datapoints whose absolute value of Z-score is greater than the threshold as outliers.

In [2]:
```
import numpy as np
outliers = []
def detect_outliers_zscore(data):
    thres = 2
    mean = np.mean(data)
    std = np.std(data)
    #print (mean, std)
    for i in data:
        z_score = (i-mean)/std
        if (np.abs(z_score) > thres):
            outliers.append(i)
    return outliers
sample_outliers = detect_outliers_zscore(sample)
print("outliers from z-score method:", sample_outliers)
```

```
outliers from z-score method: [581]
```

3.Detecting outliers using Inter Quartile Range (IQR)

Criteria:

Data points that lie 1.5 times of IQR above Q3 and below Q1 are outliers. This shows in detail about outlier treatment in python.

Steps:

1. Sort the dataset in ascending order
2. Calculate the 1st and 3rd quartiles (Q1, Q3)
3. Compute IQR = Q3 - Q1
4. Compute lower bound = (Q1 - 1.5 * IQR), upper bound = (Q3 + 1.5 * IQR)
5. Loop through the values of the dataset and check for those who fall below the lower bound and above the upper bound and mark them as outliers.

In [3]:
```
outliers = []

def detect_outliers_iqr(data):
    data = sorted(data)

    q1 = np.percentile(data, 25)
    q3 = np.percentile(data, 75)

    #print(q1, q3)

    IQR = q3-q1

    lwr_bound = q1-(1.5*IQR)
    upr_bound = q3+(1.5*IQR)

    # print(lwr_bound, upr_bound)

    for i in data:
        if (i<lwr_bound or i>upr_bound):
            outliers.append(i)

    return outliers

sample_outliers = detect_outliers_iqr(sample)

print("Outliers from IQR method: ", sample_outliers)
```

```
Outliers from IQR method:  [281, 456, 581]
```